

Autonomous AI-Powered Regulatory Compliance Micro-SaaS

The following analysis details the complete conceptualisation, technical architecture, and execution roadmap for a novel micro-Software-as-a-Service (SaaS) application. This product is designed to solve a severe, immediate, and high-value real-world problem within the South African renewable energy sector. By leveraging solely zero-cost development tools and deploying a guaranteed, zero-human-contact marketing strategy known as Programmatic SEO, this blueprint provides a definitive pathway to rapid monetisation while ensuring long-term product relevance well beyond its initial release date.

Macro-Environmental Analysis and Problem Identification

To architect an application that generates revenue without demanding aggressive outbound sales, the software must address a pain point that is both legally mandated and highly time-sensitive. The South African energy sector provides the perfect crucible for such a product.

The Small-Scale Embedded Generation (SSEG) Crisis

Between 2022 and 2024, South Africa experienced an unprecedented surge in private rooftop solar installations, driven by severe national grid instability and rolling blackouts. Hundreds of thousands of systems were installed rapidly, often outside of formal regulatory frameworks. As grid stability began to return in 2025 and 2026, municipalities, led by the City of Cape Town and the national utility provider, Eskom, initiated aggressive enforcement of Small-Scale Embedded Generation (SSEG) registration¹.

The SSEG regulatory framework governs any electricity generation installation connected to the municipal grid¹. The administrative burden for registration is immense, requiring property owners and solar installers to submit complex single-line electrical diagrams, certified inverter specifications, Certificates of Compliance (CoC), and municipal application forms². To force compliance, authorities established a hard deadline: an extension of the SSEG registration and connection fee waiver was granted only until the 30th of September 2026¹.

Following this deadline, unregistered system owners face catastrophic financial penalties. These include an unauthorised SSEG fee exceeding R6,000, the potential for immediate disconnection from the municipal electricity supply, the invalidation of household insurance claims for non-compliant electrical work, and mandated advanced metering infrastructure (AMI) smart meter upgrades that cost upwards of R10,000².

The Market Gap

For Solar Engineering, Procurement, and Construction (EPC) companies, as well as

independent electricians, processing this paperwork represents an unbillable, time-consuming administrative nightmare. They possess the technical solar quotes and hardware specifications but lack the bureaucratic efficiency to process hundreds of municipal applications rapidly. Furthermore, many systems were installed by "fly-by-night" contractors who have since disappeared, leaving homeowners stranded with complex technical invoices and no way to legalise their systems before the September 2026 deadline¹.

While enterprise-level invoice parsing solutions like Ambill, Optima, and Airparser exist for general accounts payable and financial reporting⁴, there is no consumer-accessible, niche-specific tool tailored to South African solar compliance. This constitutes a highly profitable, untapped market gap.

The Core Product Concept: The Autonomous Compliance Engine

The proposed product is an AI-driven, automated compliance engine. It operates by allowing users to upload a standard solar installation quote or invoice in any format—be it a raw PDF, a scanned image, or a forwarded email⁴.

Using advanced multimodal Large Language Models (LLMs), the application autonomously reads the document and extracts the critical hardware components, specifically isolating the inverter brand, inverter model, kilowatt capacity, and solar panel array wattage⁴. The system then algorithmically cross-references the extracted inverter model against the officially mandated NRS 097-2-1 approved equipment database². This database is the legal standard in South Africa; if an inverter is not on this list, it cannot be legally connected to the grid².

Once the hardware is validated, the application automatically maps the extracted data onto the official municipal SSEG registration PDF templates (such as those required by the City of Cape Town or eThekweni) and generates the required single-line electrical diagrams based on standard operational templates. The final output is a fully populated, ready-to-sign PDF compliance pack, returned to the user in seconds.

This product satisfies all criteria for a successful micro-SaaS: it is entirely new and creative, applying cutting-edge AI vision parsing to a stagnant bureaucratic process. It solves a highly stressful real-world problem, saving users from R10,000+ in impending fines, ensuring they will gladly pay a micro-transaction fee for the service². Furthermore, because property transfers and ongoing solar upgrades require continuous SSEG updates, the product will remain highly relevant long after the initial September 2026 deadline has passed.

Market Segmentation and Ideal Customer Profile

The application relies on a dual-sided revenue model that operates entirely through self-service portals, circumventing any need for direct sales conversations.

The primary market segment consists of South African Solar EPCs, independent solar installers, and electrical contractors. These entities are technically proficient but administratively constrained. They require bulk processing capabilities to legalise historical installations completed over the past three years¹. For an EPC managing fifty unregistered clients, manually

filling out fifty separate municipal forms is economically unviable. Offering them a portal where they can drag and drop fifty PDF invoices and instantly receive fifty completed SSEG applications provides an overwhelming return on investment.

The secondary market segment comprises individual homeowners. These individuals possess their original installation quotes but lack the technical literacy to understand terms like "maximum AC export capacity" or "anti-islanding protection." They require a frictionless, consumer-friendly interface that translates their receipt into legal compliance.

Minimum Viable Product (MVP) Definition

The Minimum Viable Product must be rigorously scoped to ensure rapid deployment. The MVP consists of a streamlined, single-page web application featuring a secure authentication portal and a drag-and-drop file upload interface.

The core operational pipeline of the MVP includes four automated phases. Initially, the system accepts the uploaded file. Subsequently, the backend utilises the AI API to parse the unstructured document and extract the hardware specifications into a structured JSON format. Following this, the application queries a proprietary database containing the scraped NRS 097-2-1 certification lists to validate the hardware⁸. Finally, a programmatic PDF generation library overlays the validated data onto a single municipal template (starting with the City of Cape Town as the initial target market) and returns the document to the user.

Selling can commence the exact moment this four-step pipeline is operational. The value proposition is binary: the user either spends hours navigating municipal bureaucracy and risking rejection, or they pay a small fee to have the software guarantee technical accuracy instantly.

Zero-Cost Technological Infrastructure Architecture

To eliminate upfront capital expenditure and financial risk, the application must be architected exclusively on platforms offering robust, perpetual free tiers. The competitive landscape of cloud computing in 2026 provides unprecedented resources for bootstrapped developers, provided the architecture avoids vendor lock-in and scales to zero when idle¹¹.

Frontend and Edge Compute: The Cloudflare Ecosystem

The frontend interface, API routing, and serverless logic will be hosted on Cloudflare Pages and Cloudflare Workers. Cloudflare provides a globally distributed serverless architecture with a highly generous free tier that outpaces competitors like Vercel and Netlify for backend-heavy processing. In 2026, Cloudflare Workers allows for 100,000 requests per day at zero cost, with a compute duration allocation of 13,000 GB-seconds per day¹². This ensures the application can handle massive, unexpected spikes in traffic without incurring hosting charges.

Furthermore, Cloudflare provides built-in DDoS protection, SSL certificates, and custom domain routing natively on the free tier¹³. Unlike platforms such as Render, where free-tier web services aggressively sleep after fifteen minutes of inactivity and require frustrating spin-up

times, Cloudflare Workers execute at the edge with near-zero cold starts¹³.

Database, Storage and Authentication: Supabase

For relational data storage, user authentication, and file storage (for temporarily holding uploaded PDFs), Supabase serves as the optimal backend-as-a-service.

The free tier includes 500 MB of PostgreSQL database storage. While 500 MB may seem limited, it is exponentially more than sufficient for storing hundreds of thousands of textual records regarding user profiles, transaction histories, and the NRS 097-2-1 inverter compliance tables¹⁵. It additionally provides 50,000 Monthly Active Users (MAUs) for authentication, 1 GB of file storage, and 5 GB of monthly bandwidth egress¹⁵.

Alternative Compute: The Koyeb Ecosystem

Should the application require long-running Docker containers or more complex backend orchestrations that exceed Cloudflare's serverless limits, Koyeb presents a highly viable alternative. Following its acquisition by Mistral AI in early 2026, Koyeb expanded its serverless platform significantly¹⁸. Koyeb's free Starter plan offers one web service and one managed PostgreSQL database, providing 1GB of RAM, 0.25 CPU, and 50 active hours for free²⁰. While Cloudflare remains superior for edge-routing and static hosting, Koyeb's native Docker support makes it an excellent fallback for deploying heavier PDF generation libraries if required¹⁹.

Backend Framework: Next.js or Encore.ts

If you want to keep things in a single framework, Next.js (deployed to Cloudflare Pages) is excellent. However, if you want a dedicated, high-performance backend, Encore.ts is a highly recommended 2026 TypeScript framework. It provides automatic infrastructure provisioning and built-in observability, offering incredibly fast performance.

Artificial Intelligence Parsing: Google Gemini API

The computational core of the application relies on interpreting highly unstructured, visually complex PDF quotes. Traditional Optical Character Recognition (OCR) is insufficient, as solar invoices vary wildly in format. Advanced multimodal LLMs are required to understand the contextual relationship between a line item, a price, and a technical specification⁴. The Google Gemini API is uniquely positioned for this requirement due to its disruptive pricing model and native document processing capabilities²¹.

As of May 2026, Google provides a free tier for its highly efficient Gemini Flash models. The Gemini 3.1 Flash-Lite and Gemini 3 Flash models offer 1,500 Requests Per Day (RPD) entirely free of charge, with context windows of up to 1 million tokens²¹. This massive context window is critical; it allows the system to ingest entire multi-page solar technical documents, including terms and conditions, without truncation.

A vital architectural consideration involves navigating the "Free Tier Illusion" inherent in Google

Cloud's 2026 billing structure. Once a credit card is attached to a Google Cloud project to enable billing, the free tier is immediately revoked, and all calls become billable from the first token²⁴. Therefore, the application must be designed using an unbilled, segregated Google Cloud project to strictly utilise the 1,500 daily free requests²⁴. Operating within this limit, the application can generate a theoretical maximum of 45,000 compliance documents per month before incurring a single cent in AI costs.

Infrastructure Evaluation Matrix

Component	Selected Provider	2026 Free Tier Limits	Commercial Upgrade Costs	Strategic Rationale
Edge Compute	Cloudflare Workers	100,000 requests/day ¹²	\$0.30 per million requests ¹²	Eliminates cold-start latency; highly superior to Render's idle-sleep mechanics ¹¹ .
Relational Database	Supabase (PostgreSQL)	500 MB DB, 50k MAU ¹⁵	\$25/month for 8 GB ²⁵	Provides robust relational mapping for compliance data; superior authentication limits compared to AWS Free Tier ²⁶ .
AI Processing	Gemini 3 Flash API	1,500 requests/day ²⁴	\$1.50 per 1M input tokens ²³	Unmatched multimodal PDF parsing; free tier far exceeds OpenAI/Anthropic limits which lack comparable

				free allocations ²² .
Alternative Backend	Koyeb	1 Web Service, 1 DB ¹⁹	\$29/month + compute ¹⁸	Excellent for Docker-based deployments if serverless execution limits are reached ¹⁹ .

Here is your 6-week sprint plan reorganized so that each developer operates as a full-stack owner of specific feature domains.

To prevent them from blocking each other, we have split the application into two core domains:

- **Developer A (The Platform Owner):** Owns the user journey, authentication, payments, UI/UX, and marketing infrastructure (Frontend + Backend for these features).
- **Developer B (The Engine Owner):** Owns the data scraping, AI extraction, validation logic, and PDF generation (Frontend + Backend for these features). Here is the aggressive 6-week vertical sprint.

Week 1: Foundations (Auth vs. Ground-Truth Data)

Developer A: Application Shell & Identity

- **Backend:** Initialize the Supabase project. Set up the `users` schema and configure Supabase Auth.
- **Frontend:** Scaffold the Next.js/React application and deploy it via Cloudflare Pages. Build the minimalist, "bureaucratic" Login and Registration UI.
- **Integration:** Connect the frontend to Supabase Auth so users can successfully create accounts and log into a blank dashboard.

Developer B: Architecture & Compliance Data

- **Backend:** Design the `approved_hardware` and `compliance_documents` schemas in Supabase. Write the ingestion scripts to programmatically scrape, clean, and format the official NRS 097-2-1 inverter list and the SAPVIA PV GreenCard registry.

- **Frontend:** Build a hidden internal admin route in the Next.js app.
- **Integration:** Display the scraped database table on the admin route to prove the data is successfully syncing from the industry bodies to the application.

Week 2: Input pipeline (Dropzone vs. AI Extraction)

Developer A: Document Intake

- **Frontend:** Build the primary dropzone UI for file uploads. Design it to project engineering competence and authority.
- **Backend:** Set up secure Supabase Storage buckets.
- **Integration:** Wire the dropzone to upload the user's raw invoice securely into their personal Supabase storage bucket, rendering a success state on the UI.

Developer B: AI Processing Engine

- **Backend:** Create the Cloudflare Worker to interact with the Gemini 3 Flash API. Execute the prompt engineering to act as a "forensic electrical auditor," forcing strict JSON output (brand, model, capacity) while ignoring financial/promotional text.
- **Frontend:** Build a temporary developer UI component.
- **Integration:** Wire the UI to pass a document buffer to the Cloudflare Worker, returning and displaying the raw, formatted JSON on the screen.

Week 3: The Validation Loop & Document Generation

Developer A: Validation Logic & Correction UI

- **Backend:** Write the logic layer utilising fuzzy string matching (Levenshtein distance algorithm) to compare Dev B's AI-extracted data against the NRS/SAPVIA Supabase tables.
- **Frontend:** Build the user verification UI. If the backend confidence threshold is too low, render a dropdown interface that forces the user to manually select the correct inverter model from the official list.
- **Integration:** Connect the AI output, the fuzzy matching, and the UI so a user can visually confirm or correct the extracted data.

Developer B: PDF Mapping Engine

- **Backend:** Integrate a Node.js PDF library. Write the mapping logic to take the

validated JSON and overlay the data points onto the official municipal SSEG registration PDF templates.

- **Frontend:** Build the "Success" UI screen and download button.
- **Integration:** Take the confirmed data from Dev A's verification step, run it through the PDF generator, and serve the final, stamped PDF to the user's browser.

Week 4: Access Control & Document History

Developer A: Document Management Dashboard

- **Backend:** Configure Row Level Security (RLS) in Supabase so users can only query their own records.
- **Frontend:** Build the "Past Documents" interface inside the user dashboard.
- **Integration:** Fetch the user's previously generated SSEG PDFs from the database and render them as downloadable items in the frontend dashboard.

Developer B: Edge-Case Handling & Engine Polish

- **Backend:** Refine the Gemini prompts and Levenshtein thresholds by testing against highly messy, real-world solar quotes containing typographical errors.
- **Frontend/Integration:** Improve the error-handling UI. If a document is completely illegible or non-solar related, ensure the backend gracefully catches it and the frontend displays a clear, actionable error message rather than crashing.

Week 5: Monetisation Mechanics

Developer A: Payment Gateway (Full-Stack)

- **Backend:** Integrate the payment processor (PayFast for local, Stripe for international). Build the webhooks to listen for successful transactions and update a user's credit bundle in Supabase. Write the logic granting the user their first document for free.
- **Frontend:** Build the checkout modal, paywall UI, and credit balance indicator in the dashboard.
- **Integration:** Lock the final "Download PDF" button behind a database check. If the user has 0 credits (and has used their free hook), trigger the frontend checkout modal.

Developer B: Infrastructure Security & Rate Limiting

- **Backend:** Secure the Cloudflare Workers. Implement strict rate limiting to prevent abuse of the Gemini API and the PDF generation endpoints.
- **Frontend/Integration:** Ensure the frontend handles rate-limit HTTP errors by informing the user they are sending too many requests.

Week 6: The Zero-Contact Marketing Engine

Developer A: Programmatic SEO (pSEO) Deployment

- **Backend:** Write the Next.js routing scripts to query the Supabase database of approved inverters and cross-reference them with target municipalities.
- **Frontend:** Design the dynamic Next.js page template, optimizing the H1 headers and metadata for high-intent search permutations (e.g., "SSEG Registration for [Brand] in [Municipality]").
- **Integration:** Generate thousands of these static HTML landing pages at build time on Cloudflare Pages, ensuring every page points directly to the core dropzone.

Developer B: Production Handoff & Launch Ops

- **Backend:** Finalize environment variables, switch all API keys (Gemini, PayFast, Supabase) from test/dev to live production environments.
- **Frontend/Integration:** Conduct full end-to-end user testing. Upload a messy quote, pass the AI extraction, trigger manual validation, process a real payment, and successfully download the SSEG PDF.

Zero-Contact Marketing Strategy: Programmatic SEO (pSEO)

The requirement for a marketing strategy that guarantees user acquisition without manual sales outreach, networking, or human interaction is solved entirely by Programmatic SEO (pSEO). Traditional outbound sales are labour-intensive and scale poorly. pSEO, conversely, leverages automation to generate thousands of highly specific, search-engine-optimised landing pages based on a structured dataset²⁸. Software companies like Zapier and HubSpot built their user bases using this exact methodology³⁰. Because the application relies on the NRS 097-2-1 compliance

database containing hundreds of specific inverter models⁸, it possesses a structural pSEO advantage³⁰.

The Strategy Mechanics

When a solar installer is tasked with registering a specific hardware setup, they do not search for generic terms like "solar compliance software." Instead, they search for highly specific, bottom-of-the-funnel, long-tail keywords indicating immediate commercial intent³¹. For example, they will query Google with: *"How to register a Sunsynk 8kW Hybrid Inverter for SSEG in Cape Town."*

The marketing strategy relies on capturing this exact traffic pattern. Using the structured data residing in the Supabase backend, the developer will construct a template that dynamically generates a unique URL and page for every possible combination of variables²⁹.

The Keyword Matrix Implementation

A programmatic keyword matrix combines head terms with modifiers to cover the entire search landscape²⁹.

- **Variable 1 (The Hardware):** Deye, Sunsynk, Growatt, SMA, Victron, Huawei (derived directly from the NRS 097-2-1 database)⁸.
- **Variable 2 (The Municipality):** City of Cape Town, City of Johannesburg, eThekweni, Nelson Mandela Bay¹.
- **Variable 3 (The Action):** SSEG Registration, Commissioning Form, Compliance Certificate.

Through this matrix, multiplying 300 approved inverter models by 10 major municipalities yields 3,000 highly targeted landing pages generated instantly.

Template Architecture and Conversion Optimisation

Each generated page must provide genuine value to satisfy search engine algorithms and avoid being flagged as thin, AI-generated spam²⁹. The template will dynamically inject:

1. A localised H1 header (e.g., *SSEG Registration Guide for Sunsynk 8kW in Cape Town*).
2. Verified technical data pulled from the database regarding that specific inverter (e.g., maximum AC output, anti-islanding certification status).
3. A summary of the specific municipality's September 2026 deadline and impending fine structures².
4. The Conversion Mechanism: A prominently displayed upload zone stating, *"Upload your Sunsynk 8kW quote here to auto-generate your completed Cape Town registration forms in 30 seconds."*

Because these highly specific queries have near-zero competition, the pages will index rapidly. Searchers landing on these pages possess immediate commercial intent, allowing the system to acquire users passively, entirely fulfilling the zero-contact marketing requirement.

Financial Modelling and Unit Economics

The economic viability of this application relies on achieving near-perfect gross margins by exploiting the asymmetric cost advantages of modern cloud free tiers.

Mathematical Cost Analysis

Let the operational cost per user transaction be defined mathematically:

$$\text{Total Cost} = C_{\text{compute}} + C_{\text{database}} + C_{\text{AI}}$$

Given the architectural choices:

- C_{compute} (Cloudflare) remains R0 up to 100,000 daily requests¹².
- C_{database} (Supabase) remains R0 as long as the total data remains under 500 MB and monthly active users remain under 50,000¹⁵.
- C_{AI} (Gemini 3 Flash) remains R0 up to 1,500 parsing requests per day²⁴.

Consequently, the marginal cost of producing one compliance document is functionally R0.

Revenue Model and Profitability

The recommended monetisation strategy is a pay-per-use model utilising a credit system, which is highly effective for B2B utility software.

1. **The Hook:** Users receive one free document generation to prove the AI's accuracy and build trust.
2. **The Transaction:** Users purchase "Compliance Credits." For example, 10 credits for R500 (R50 per document).
3. **The Value Proposition:** A typical residential solar installation costs between R75,000 and R250,000³³. An electrician's hourly labour rate ranges from R450 to R850³⁶. Paying R50 to eliminate two hours of bureaucratic form-filling—and avoiding a potential R6,000 municipal fine²—represents an overwhelming and undeniable return on investment for the installer.

Operating strictly under the free tier limit of 1,500 requests per day, if even a conservative 5% of daily AI capacity is utilised for paid conversions (75 documents per day) at a rate of R50 per document, the application generates R3,750 per day. This

equates to roughly R112,500 per month in pure profit, achieved with zero infrastructure expenditure and zero human sales intervention.

Future-Proofing and Horizontal Expansion

While the September 2026 SSEG deadline provides the immediate catalyst for rapid adoption, relying on a single regulatory event poses long-term retention risks. To ensure the application remains viable indefinitely, the underlying technology—AI-driven document parsing mapped to regulatory compliance frameworks—must be applied to adjacent localised problems. The architecture built for solar compliance is entirely modular and can be pivoted to other high-friction industries overnight.

Expansion Vector 1: The Water Security Crisis

Parallel to the energy crisis, South Africa is currently experiencing severe municipal water shortages, often termed "water-shedding"³⁷. The national government has acknowledged massive infrastructure failures, with half of the country's treated water lost to leaks before reaching consumers³⁹. Consequently, businesses and homeowners are rapidly installing bulk water backup systems, primarily relying on JoJo tanks. The costs for these systems are substantial. A standard residential water tank setup costs between R7,000 and R20,000, while fully pressurised, multi-tank systems connected to the municipal mains range from R40,000 to over R80,000⁴⁰. Commercial and industrial storage solutions frequently exceed R100,000⁴⁰. Plumbers charge between R2,500 and R6,000 purely for installation labour, with complex jobs taking several days³⁶.

Just as solar systems require municipal sign-off, pressurised water systems integrated into municipal mains increasingly require plumbing compliance certificates and complex financing approvals. The AI parser can be trained to ingest plumbing quotes, identify the tank capacity (e.g., 5250L JoJo Tank)⁴², the booster pump specifications, and the filtration systems, automatically generating the necessary compliance forms or detailed financing applications for the homeowner.

Expansion Vector 2: POPIA and PAIA Compliance Automation

The Protection of Personal Information Act (POPIA) and the Promotion of Access to Information Act (PAIA) represent ongoing, permanent regulatory burdens for all South African businesses⁴³. Every private entity, regardless of size, is legally required to maintain a PAIA Manual and register an Information Officer with the Information

Regulator⁴³. Websites failing to maintain compliant privacy policies, cookie banners, and PAIA manuals face catastrophic administrative fines of up to R10 million or criminal liability⁴⁴.

The solar installers and plumbers utilising the primary application invariably operate their own commercial websites. Post-launch, the application can cross-sell an automated PAIA Manual and POPIA compliance generator. By having the contractor upload basic company information (or by extracting it autonomously from the letterheads of their previously uploaded invoices), the system can instantly generate bespoke, legally compliant manuals and privacy policies that typically cost upwards of R20,000 when commissioned through traditional law firms⁴⁵.

Expansion Vector 3: The CBAM Carbon Tax Gateway

Looking globally, starting permanently in 2026, the European Union's Carbon Border Adjustment Mechanism (CBAM) requires all manufacturers exporting goods to the EU to rigorously report the embedded carbon emissions of their products⁴⁸. This severely impacts African exporters in carbon-intensive sectors such as cement, iron, steel, aluminium, and fertilisers⁴⁸.

The technical architecture built for the solar quoting tool can be effortlessly pivoted to address this international trade barrier. Instead of uploading solar quotes, African manufacturing SMEs upload their energy bills and material procurement invoices. The Gemini AI extracts the volumetric consumption data, applies localised carbon emission factors, and algorithmically generates the required XML reports for the EU CBAM Declarant Portal⁵². This pivot transitions the micro-SaaS from a localised South African compliance tool into a highly lucrative, continent-wide industrial export gateway, tapping into enterprise-level budgets.

Synthesis

By identifying the severe friction point between rapid renewable energy adoption and rigid municipal bureaucracies, this micro-SaaS concept guarantees deep market resonance upon release. Utilising an architecture built entirely on the generous free tiers of Cloudflare, Supabase, and the Gemini API ensures absolutely zero financial risk during the development and scaling phases.

Furthermore, by weaponising Programmatic SEO driven by publicly available hardware compliance databases, the platform ensures highly targeted user acquisition without requiring any manual marketing, networking, or human interaction. Executing the provided six-week roadmap will position the application perfectly to capture the panic-driven market demand leading up to the September 2026 regulatory deadline. By subsequently pivoting the underlying AI parsing technology to adjacent crises in water infrastructure, data privacy, and international carbon taxation, the developer

secures a highly profitable, self-sustaining software asset with guaranteed longevity.

Functional Requirements

1. User Authentication & Account Management

- **Sign-Up and Authentication:** The system must allow users to register and log in securely using email/password or OAuth (Google).
- **Role-Based Profiles:** The system must differentiate between user types (Individual Homeowners vs. Solar EPC Installers) during onboarding to tailor the dashboard view.
- **Credit Balance Tracking:** The system must maintain and display a real-time count of available "Compliance Credits" on the user's dashboard.

2. Document Ingestion & Storage

- **Drag-and-Drop File Upload:** The system must accept user file uploads via a drag-and-drop web interface supporting PDF, JPEG, and PNG formats up to 10MB.
- **Bulk Uploading (B2B):** The system must allow Solar EPC users to select and upload multiple invoice files simultaneously for queue processing.
- **Secure Temporary Storage:** The system must securely store uploaded raw files and generated PDFs in private user-isolated storage buckets with strict Row Level Security (RLS).
- **Document History Logs:** The system must maintain a history log of all past uploads, execution statuses (Success/Failed), and downloadable generated outputs for each user.

3. AI Data Extraction Engine

- **Multimodal Parsing:** The system must pass file buffers to a multimodal LLM API to read and extract technical text from unstructured layouts, low-res scans, and multi-page documents.
- **Structured Schema Output:** The AI engine must format extracted data into a rigid JSON structure containing exactly: Inverter Brand, Inverter Model Number, Inverter kW Capacity, and Total Solar Panel Array Wattage.
- **Confidence Scoring:** The parsing engine must calculate a validation confidence

score based on text clarity and structural completeness.

4. Hardware Validation & Cross-Referencing

- **Database Querying:** The system must automatically query a local database containing the official NRS 097-2-1 approved inverter list using the AI-extracted data.
- **Fuzzy Matching Logic:** The system must utilize fuzzy string matching algorithms (e.g., Levenshtein distance) to match variations in handwritten or mistyped model names against the official database.
- **Compliance Status Flags:** The system must label the hardware status as **Approved** (exact/close match found), **Not Approved** (explicitly forbidden), or **Unverified** (confidence below threshold).

5. Human-in-the-Loop (HITL) Review Screen

- **Side-by-Side Verification UI:** The system must render a data-review screen displaying the original uploaded document next to editable text fields pre-filled with the AI's JSON output.
- **Manual Overrides:** The system must allow users to manually correct any misidentified hardware specifications, project locations, or client details prior to final generation.
- **NRS 097-2-1 Selection Dropdown:** For **Unverified** hardware, the system must provide a searchable dropdown list pulling directly from the approved database to let the user select the correct model manually.

6. PDF Compliance Pack Generation

- **Form Mapping & Overlay:** The system must programmatically map the verified JSON variables onto the coordinate fields of official municipal SSEG registration PDF forms (starting with the City of Cape Town).
- **Single-Line Diagram Generation:** The system must select and append a standard single-line electrical diagram template that corresponds directly to the verified inverter model and system size.
- **Download Delivery:** The system must compile the filled forms and diagrams into a unified, flattened, print-ready "Compliance Pack" PDF and prompt an immediate browser download.

7. Monetization, Billing & Credit Gates

- **First-Time Free Trial Hook:** The system must automatically grant exactly one (1) free compliance credit to newly registered accounts upon email verification.
- **Payment Gateway Checkout:** The system must integrate local payment processors (PayFast/Stripe) to process web payments for single credits or bulk discount credit bundles.
- **Webhook Verification:** The system must listen for secure payment gateway webhooks to instantly credit user balances upon successful transaction clearance.
- **Execution Credit Gate:** The system must check the user's credit balance before executing the PDF mapping phase, deducting exactly one credit per generated pack, and blocking generation if the balance is zero.

8. Programmatic SEO (pSEO) Page Router

- **Dynamic Landing Page Generation:** The system must use a static-site template to compile unique public landing URLs at build time for every permutation of [Inverter Brand] + [Inverter Model] + [Municipality].
- **Local Data Injection:** Each programmatic page must pull dynamic text from the backend database detailing specific municipal deadline penalties, fine warnings, and hardware certification data.
- **Inline Upload Entrypoint:** Every programmatic landing page must feature a functional document upload component that feeds files straight into the user onboarding/processing pipeline.

Non-Functional Requirements

1. Performance and Scalability

- **Near-Zero Cold Starts:** The backend API must execute on edge infrastructure (Cloudflare Workers) to ensure API response times initiate within less than 50 milliseconds, avoiding the idle-sleep delays common in standard free-tier hosting.
- **Total Processing Throughput:** The system must complete the end-to-end processing pipeline—from document upload through AI extraction, database validation, and PDF compilation—in less than 15 seconds per single document under normal network conditions.
- **Concurrency Handling:** The serverless architecture must support scaling from

zero to thousands of simultaneous executions without manual infrastructure adjustment or degradation of processing speeds.

2. Cost and Infrastructure Constraints

- **Zero-Dormant Hosting Costs:** The entire production stack must utilize perpetual free tiers (Cloudflare, Supabase, Google Gemini) to guarantee that operational hosting costs scale to absolutely R0 when the platform is idle.
- **AI API Resource Budgeting:** The backend must restrict unbilled API transactions to a maximum threshold of 1,500 requests per day to remain strictly within the free-tier constraints of the Google Gemini API.
- **Database and Storage Caps:** The system's relational data usage must not exceed 500 MB of PostgreSQL storage, and user file storage (PDFs/images) must be capped below 1 GB at any given time via automated data retention sweeps.

3. Security and Data Protection

- **Data Isolation (Row Level Security):** The relational database and storage buckets must enforce strict PostgreSQL Row Level Security (RLS) policies, making it mathematically impossible for a user to query, view, or download documents belonging to another user.
- **End-to-End Encryption:** All data in transit must be encrypted using mandatory HTTPS/TLS 1.3 protocol layers managed automatically at the network edge.
- **Data Retention & Ephemerality:** To minimize storage usage and protect customer privacy, uploaded raw invoices and generated compliance packs must be permanently scrubbed from the storage buckets exactly 48 hours after generation.
- **Regulatory Alignment (POPIA):** The data collection pipelines must adhere strictly to South Africa's Protection of Personal Information Act (POPIA). Financial figures, bank details, or unrelated personal information extracted by the AI from invoices must be immediately dropped from memory and never committed to the database.

4. Availability and Reliability

- **Uptime Objective:** The core user dashboard and document processing pipelines must target a 99.9% uptime availability, relying on the globally distributed nature of the edge hosting network.
- **Graceful AI Failures:** If the AI parsing engine fails, times out, or hits its daily free-tier request cap, the system must fail gracefully by displaying a clear, polite notification to the user and pausing credit deductions, rather than throwing an unhandled application crash.

5. Maintainability and Extensibility

- **Shared Type Safety:** The platform must utilize a unified TypeScript monorepo architecture (Turborepo) to ensure that changes made to the database schema or AI output data structures automatically propagate to the frontend, preventing version mismatch bugs.
- **Decoupled PDF Templating:** The logic mapping data variables to municipal forms must be abstracted into modular configuration files rather than hardcoded. This ensures that when a municipality updates an application form, a developer can update the layout mapping coordinates in minutes without refactoring the core backend application code.

6. Usability and Accessibility

- **Mobile-Responsive Interface:** The web application must be entirely responsive, allowing field technicians and solar engineers to log in, snap a photo of an invoice on a smartphone from a physical job site, upload it, and view the review dashboard easily.
- **Zero-Training UI:** The interface must be designed to look authoritative and highly professional, minimizing user cognitive load by utilizing a clear, guided step-by-step progress flow.

Database Schema & Architecture

Document Purpose: This document details the complete PostgreSQL relational database schema for the Autonomous Compliance Engine, designed specifically for a Supabase backend. It translates the application's functional, security, and monetization requirements into strict data logic.

1. Custom Enumerated Types (Enums)

Enums enforce strict data types at the database level, ensuring the AI or frontend cannot inject invalid statuses into your system.

2. Core Tables and Documentation

2.1 The `users` Table

Supabase handles the actual secure password hashing and authentication in a hidden `auth.users` schema. This public `users` table extends that core identity with your specific application logic, namely role-based access and the monetization mechanics.

Field Name	Type	Purpose / Requirement
<code>id</code>	<code>UUID</code>	Links directly to Supabase's internal auth system. Cascades on delete.
<code>email</code>	<code>TEXT</code>	Primary contact and login identifier.
<code>role</code>	<code>user_role</code>	Differentiates between 'homeowner', 'epc_installer', and 'admin'.
<code>credit_balance</code>	<code>INTEGER</code>	Tracks "Compliance Credits". Defaults to 1 for the "First-Time Free Trial Hook".
<code>created_at</code>	<code>TIMESTAMPTZ</code>	Standard audit timestamp.
<code>updated_at</code>	<code>TIMESTAMPTZ</code>	Standard audit timestamp.

2.2 The **approved_hardware** Table (The Ground Truth)

This is the programmatic representation of the NRS 097-2-1 and SAPVIA lists. The AI's extracted data is cross-referenced against this legal standard. It also serves as the data source for your Programmatic SEO (pSEO) engine.

Field Name	Type	Purpose / Requirement
id	UUID	Unique identifier for each hardware piece.
brand	TEXT	E.g., 'Sunsynk', 'Deye'. Used for pSEO keyword matrices.
model_number	TEXT	Exact alphanumeric string. Must be UNIQUE for fuzzy matching logic.
kw_capacity	NUMERIC	Technical output capacity of the inverter.
anti_islanding_certified	BOOLEAN	Critical compliance check for municipal sign-off.

2.3 The **compliance_documents** Table (The Engine Core)

This table is the transactional heart of the application. It tracks the state of the document pipeline from the moment a user drops a file through AI extraction, human validation, and final PDF generation.

Field Name	Type	Purpose / Requirement
id	UUID	Unique transaction ID.
user_id	UUID	Ties document to user. Crucial for Row Level Security (RLS) isolation.
raw_file_path	TEXT	Points to the original PDF/JPEG uploaded to Supabase Storage buckets.
generated_pdf_path	TEXT	Points to the final, completed "Compliance Pack" PDF.
extracted_brand	TEXT	Populated by the Gemini AI's JSON output.
extracted_model	TEXT	Populated by the Gemini AI's JSON output.
extracted_capacity	NUMERIC	Populated by the Gemini AI's JSON output.
extracted_panel_wattage	NUMERIC	Populated by the Gemini AI's JSON output.

status	document_status	Tracks execution flow (e.g., 'processing', 'completed').
validation_confidence	NUMERIC	The confidence score from the AI extraction phase.
matched_hardware_id	UUID	Foreign key linking AI text to ground-truth approved_hardware table.
hardware_compliance	compliance_status	Labels status as 'Approved', 'Not Approved', or 'Unverified'.
expires_at	TIMESTAMPTZ	POPIA Mandate: Hardcodes deletion exactly 48 hours after creation.

2.4 The **payment_transactions** Table

Required to listen to webhooks from PayFast/Stripe and create an immutable ledger of credit purchases to top up the user's balance safely.

Field Name	Type	Purpose / Requirement
id	UUID	Unique transaction ID.
user_id	UUID	The user who made the purchase.
gateway_reference	TEXT	Unique ID from PayFast/Stripe. Prevents double-crediting.
amount_paid	NUMERIC	The monetary value of the transaction.
credits_added	INTEGER	Number of credits added to the user's account.
created_at	TIMESTAMPTZ	Timestamp of the purchase.

3. Row Level Security (RLS) Implementation

To mathematically guarantee data isolation and adhere to POPIA, these RLS policies must be applied directly to the Supabase tables so users cannot view each other's documents.

-- Enable RLS on all tables

-- 1. Users can only view and edit their own profile

-- 2. Users can only view and manage their own documents

-- 3. Hardware database is strictly public read-only (needed for pSEO and dropzone)

4. Supabase Storage Buckets

Two isolated buckets are required to handle files, enforcing the same RLS policies as the database.

1. **raw_uploads**: Accepts PDF/JPEG/PNG up to 10MB. RLS policy ensures a user can only upload/read files matching their `auth.uid()`.
2. **compliance_packs**: Stores the final, flattened PDF mapped with the single-line electrical diagram.

5. Automated Data Ephemerality (POPIA Compliance)

To ensure raw invoices and generated documents are scrubbed after exactly 48 hours (minimizing the 1GB free storage limit and complying with POPIA), implement a Supabase `pg_cron` job:

SQL

```
-- Automatically deletes database records older than 48 hours
-- Supabase Storage triggers will listen to this deletion and scrub the actual files from the buckets.
SELECT cron.schedule(
  'purge-old-documents',
  '0 * * * *', -- Runs every hour
  $$ DELETE FROM public.compliance_documents WHERE expires_at <= NOW(); $$
);
```

1. Architectural Patterns & Design

The "Zero-Cost" mandate strictly ties us to edge compute (Cloudflare Workers) and a backend-as-a-service (Supabase). This dictates a **Serverless Event-Driven Monolith**.

- **The Approach**: Next.js handles the frontend routing and programmatic SEO (pSEO) generation, while Cloudflare Workers handle the heavy API routing and Gemini AI interaction.
- **Pros**: Infinite scalability without manual intervention. Cloudflare executes at the edge with near-zero cold starts, drastically outperforming standard containerized environments.
- **Cons**: We are fundamentally locked into the Cloudflare/Supabase ecosystem. Debugging edge functions locally can sometimes exhibit subtle differences compared to the production edge environment.
- **Design Patterns**:
 - **Strategy Pattern**: For AI fallback logic. If Gemini 3 Flash times out or hallucinates the JSON structure, the system should instantly swap to an

alternative prompt strategy or a smaller, faster model (like Gemini Flash-Lite) to retry before failing.

- **Observer/Pub-Sub Pattern:** Document processing is async. The frontend should subscribe to a Supabase real-time channel. When the backend completes the PDF, it publishes an event, automatically updating the user's UI.

2. Deployment Requirements & IaC

Managing branches strictly through Git with rigorous ESLint pre-commit hooks ensures that bad code never reaches the automated pipeline. While standard Next.js deployments naturally gravitate toward Vercel, the system's reliance on 100,000 daily free edge requests necessitates Cloudflare Pages and Workers.

- **Environment Parity:**
 - **development:** Local Wrangler CLI and Supabase local emulator.
 - **staging:** Auto-deployed via Git on every pull request to a preview branch.
 - **production:** The **main** branch deployed to the custom domain.
- **IaC (Infrastructure as Code):**
 - **Decision:** Use **wrangler.toml** for Cloudflare and Supabase Migrations (**supabase/migrations**) for the database schema.
 - **Pros:** Total environment reproducibility. No "click-ops."
 - **Cons:** Higher learning curve for new developers compared to UI-driven setup.
- **Secrets Management:**
 - **Decision:** Inject **GEMINI_API_KEY**, **PAYFAST_SECRET**, and Supabase keys via Cloudflare Worker Secrets and GitHub Actions.
- **Rollback Strategy:**
 - **Decision:** Commit hash pinning. If a deployment corrupts the PDF mapping logic, we rollback directly in the Cloudflare dashboard to the previous successful Git deployment hash. Time to resolution: < 30 seconds.

3. Caching & Network Optimization

Given that field technicians might be uploading heavy 10MB PDFs from mobile devices with poor reception, optimizing transport-layer delivery utilizing HTTP/3 and QUIC over Cloudflare's edge will significantly reduce connection handshake overhead compared to legacy TCP.

- **Database Caching (Cloudflare KV):** The official NRS 097-2-1 approved hardware list is entirely static. Querying Supabase every time the AI needs to fuzzy-match an inverter is wasteful. We will cache this entire table in Cloudflare KV.
 - **Pros:** Sub-10ms reads globally.
 - **Cons:** Requires a background cron job to invalidate and update the KV store when the national database updates.
- **Frontend Caching:** Apply **stale-while-revalidate** (SWR) for the Next.js UI components.

4. UI/UX "Must-Haves"

Using the *ComplyStack Brand Guidelines* (Compliance Navy #14213D, Paper White #FAFAFA, Signal Teal #0686A4), the UI needs to project bureaucratic authority while feeling instant.

- **Skeleton Loaders:** When a user opens their dashboard, render structural skeletons of their past documents rather than generic spinners. This reduces perceived latency.
- **Optimistic UI:** When a user initiates a drag-and-drop upload, instantly show the document in a "Processing" state on the dashboard. If the backend fails, revert the state and show a toast error.
- **Tooltips:** Solar terminology is complex for homeowners. Hoverable tooltips must be present on the Human-in-the-Loop (HITL) review screen to explain terms like "Anti-islanding protection" or "Maximum AC export".

5. Operational Telemetry (The Engine Room)

- **Structured Logging:**
 - **Decision:** Use `Pino.js` to output strict JSON logs containing `trace_id`, `user_id`, and `execution_ms`.
 - **Pros:** Instantly searchable in Datadog/Papertrail.
 - **Cons:** JSON logs are hard to read in a raw terminal without a formatter like `pino-pretty`.
- **Error Monitoring:**
 - **Decision:** Sentry integrated at both the Next.js boundary and the Cloudflare Worker runtime.
 - **Actionable Setup:** Configure Sentry to ignore 404s to save quota, but alert the Slack/Discord channel instantly on any 500-level error involving the payment gateway or AI parser.
- **Rate Limiting:**
 - **Decision:** Use Cloudflare's native Rate Limiting rules on all public endpoints.
 - **Threshold:** Restrict `/api/process` to 10 requests per minute per IP to prevent a single bad actor from burning through our 1,500 free Gemini API requests.
- **Health Check Endpoints:**
 - **Decision:** Create a lightweight `/health` route. It pings Supabase (`SELECT 1`) and verifies the Cloudflare Worker memory state